

# 面向 AI Agent 软件开发的

## RFLP 统一语义建模与模型驱动执行方法研究

### 研究设想与技术方案报告

#### 摘要

当前 AI Agent 辅助软件开发主要依靠需求文档、架构规划、接口说明、任务清单和多轮对话维持工程上下文。随着项目持续演进，不同时间、不同 Agent 生成的文档容易出现术语不一致、模块重复、接口漂移、需求与实现脱节以及架构决策无法追溯等问题。现有长上下文或 RAG 方法可以改善文档检索，但难以提供稳定的工程对象身份、统一的系统语义和可计算的架构状态。

本文提出一种面向 AI Agent 软件开发的 RFLP 统一语义建模与模型驱动执行方法。该方法以 SysML v2 作为权威模型表达和操作语言，在自然语言工程文档与 RFLP 模型之间建立工程语义编译层，将多源 Agent 文档转换为带有来源、类型、关系、版本和置信度的语义断言，并依次构建需求层、功能层、逻辑层和物理实现层。系统在逻辑层生成多套功能分配和组件交互方案，在物理层从组件库中搜索具体实现配置，通过约束满足、多目标权衡及敏感性分析形成架构基线。随后，系统将目标 RFLP 模型与代码反向抽取形成的实际实现模型进行比较，根据模型差异生成带有修改范围、前置条件、架构约束和验收标准的开发任务契约，驱动编码 Agent 和测试 Agent 执行，并将实现与验证证据回写模型。

本文的研究重点不是单纯利用大模型生成 SysML v2 文本，而是建立“多源文档—统一语义—RFLP 设计综合—方案决策—Agent 任务—代码与验证证据”的闭环，为长期、多轮、多人或多 Agent 软件开发提供系统工程控制层。

**关键词：**AI Agent；MBSE；SysML v2；RFLP；工程语义编译；模型驱动开发；软件架构权衡

#### 核心技术主线

多源 Agent 文档 → 可追溯语义断言 → RFLP 分层综合 → L/P 候选空间 → 约束求解与 Trade Study  
→ 目标实现模型求差 → TaskContract → Agent 执行与证据回写。

## 1. 研究背景与问题定义

当前 AI Agent 在软件开发中会持续生成产品需求、系统架构、模块职责、API 定义、数据库设计、任务计划和测试记录。这些工程信息通常分布在不同文档、会话和 Agent 上下文中，即使全部被存档，也缺少稳定对象身份、正式关系和权威版本。

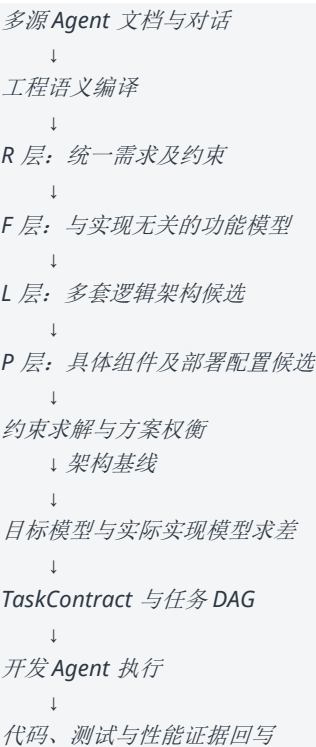
文档A：用户认证模块负责登录和JWT 签发。  
文档B：AccountService 负责登录和会话管理。  
文档C：IdentityService 提供Token 生成接口。

上述描述可能对应同一个组件，也可能反映职责边界变化。普通文档系统或 RAG 系统只能提高检索命中率，不能稳定判断不同名称是否指向同一工程对象，当前哪个接口定义属于已批准基线、某项内容处于 R/F/L/P 哪一层，以及某个变更将影响哪些代码和测试。

因此，本研究拟在自然语言规划文档和开发 Agent 之间增加一个 MBSE 系统工程层，将分散、概率化、易漂移的文本工程信息转换为统一、可追溯和可计算的 RFLP 模型。

## 2. 研究目标与总体架构

研究目标是建立从多源 Agent 文档到 RFLP 目标模型、从候选架构到可解释基线、再从目标/实现模型差异到可执行 Agent 任务的闭环。



### 2.1 三类模型与数据职责

| 模型/数据层     | 主要内容                                                 | 职责                             |
|------------|------------------------------------------------------|--------------------------------|
| 原始证据与语义断言层 | SourceArtifact、TextSpan、SemanticClaim、ConflictRecord | 保存“谁在何时基于什么来源提出了什么”，不直接覆盖权威模型。 |

|            |                                                                              |                                  |
|------------|------------------------------------------------------------------------------|----------------------------------|
| 目标 RFLP 模型 | Requirement、Function、Logical/<br>Physical Component、Interface、<br>Constraint | 表达系统“应该是什么”，形成设计基线。              |
| 实际实现模型     | 代码模块、API、Schema、依赖、部署、<br>测试                                                 | 表达系统“现在实际是什么”，由代码<br>与运行制品确定性抽取。 |

**任务生成的基本关系** 目标设计模型 - 实际实现模型 = 待完成、待修复、待验证  
或待评审的 engineered work。

2.2 核心子系统

- 工程制品接入层：解析对话、Markdown、Word、API 文档、代码和测试结果，并保存结构化来源定位。
- 工程语义编译器：完成原子化、语义帧抽取、实体对齐、冲突检测和模型补丁生成。
- RFLP 模型综合器：构建 R 层和 F 层，生成 L 层候选，并完成 P 层组件配置。
- SysML v2 权威模型仓库：维护稳定对象身份、版本、分支、基线和跨层追溯。
- 方案权衡引擎：进行硬约束过滤、Pareto 分析、多指标排序和敏感性分析。
- 模型差异与任务引擎：比较目标模型和实际实现，生成 TaskContract 与 DAG。

3. 工程统一语义模型

3.1 工程语义中间表示 ESIR 在自然语言和 SysML v2 之间增加工程语义中间表示 ESIR，避免

LLM 直接生成或修改整份权威

模型。ESIR 同时承担训练数据接口、人工审核接口和模型投影接口。

SourceArtifact  
TextSpan  
SemanticClaim  
Requirement / Constraint  
Function  
LogicalComponent  
PhysicalComponent  
Interface / DataDefinition  
ArchitectureCandidate / Decision  
Task / ImplementationArtifact  
VerificationCase / Evidence / Baseline

自然语言中的每项工程结论首先转换为 SemanticClaim。Claim 表示来源中的断言，RFLP 模型元素表示当前被批准的系统事实。二者分离后，可以同时保留互相矛盾的 Agent 输出，而不破坏主模型。

```
{
  "claim_id": "CLM-1024",
  "source_artifact": "DOC-008",
  "source_span": "section-3.2:paragraph-4",
  "claim_type": "interface_definition",
  "subject": "AuthenticationService",
  "predicate": "provides",
  "object": "LoginInterface",
```

```
"modality": "required",
"confidence": 0.91,
"producer": "architecture-agent",
"status": "candidate"
}
```

**3.2 稳定对象身份与别名**所有模型元素使用稳定 UUID，名称和描述仅作为可变属性。实体对齐确认后，不同文档中的 *AuthService*、*AccountService* 和用户认证模块可映射到同一对象。

```
UUID: LCOMP-008
标准名称: Authentication Service
别名: AuthService / AccountService / 用户认证模块
```

3.3 跨层正式关系

```
Requirement satisfiedBy Function
Function allocatedTo LogicalComponent
LogicalComponent realizedBy PhysicalComponent
Component provides / requires Interface
Interface transfers DataDefinition
PhysicalComponent implementedBy ImplementationArtifact
Requirement verifiedBy VerificationCase
VerificationCase supportedBy Evidence
Task realizes ModelElement
```

关系必须具有明确语义，不能统一退化为related\_to，否则无法执行覆盖检查、影响传播、约束求解和任务生成。

4. 自然语言语义识别与结构化

**4.1 文档结构解析与来源定位**文档接入后应建立结构树，不直接按固定 Token 切块。Markdown 解析 AST；Word 保留标题、段落、列表和表格；Agent 对话额外保存 conversation\_id、message\_id、agent\_role、timestamp、parent\_message 和 tool\_result。所有抽取对象必须关联原始 TextSpan。

```
Document
├─ Section
├─ Paragraph
├─ ListItem
├─ TableRow / TableCell
├─ CodeBlock
└─ TextSpan
```

4.2 工程片段分类与原子化

首先将文本片段分类为 Requirement、Constraint、Assumption、FunctionDescription、LogicalArchitecture、PhysicalSelection、InterfaceDefinition、DataDefinition、DesignDecision、TaskPlan 或 VerificationResult。原型阶段采用 LLM Structured Output 与 Pydantic Schema；积累标注后以专用 Transformer 替代高频任务。

复合需求需要拆为单一义务动作。数量、时延、准确率等转为独立 Constraint；模糊表达进入 AmbiguityRecord；隐含推导与原文明确陈述分开保存。

原文：系统应自动识别角色，允许用户修改识别结果，并保留全部历史版本。

- R1：系统应自动识别角色。
- R2：系统应允许用户修改角色识别结果。
- R3：系统应保存角色识别结果的历史版本。

4.3 工程语义帧原子片段进一步映射为工程语义帧，字段包括 Actor、Action、Object、Input、Output、Trigger、Precondition、Postcondition、Constraint、QualityAttribute、Modality、Priority 和 AcceptanceCriterion。

```
{
  "subject": "System",
  "modality": "shall",
  "action": "retain",
  "object": "CharacterDataVersion",
  "condition": "after each accepted modification",
  "verification_intent": "restore any retained version"
}
```

4.4 实体对齐实体对齐采用“召回—重排—决策”三级结构。召回阶段同时使用标准名称、别名词典、BM25、Embedding、元素类型过滤、RFLP 层过滤和关系邻域检索；重排阶段综合名称、描述、能力、接口、上下游关系和来源上下文。

| 决策类型     | 语义                  |
|----------|---------------------|
| MERGE    | 确认新片段与现有元素是同一对象。    |
| REFINE   | 新片段细化现有对象的职责、约束或接口。 |
| ALIAS    | 新名称是已有对象的别名。        |
| CONFLICT | 新定义与现有基线冲突。         |

|        |                         |
|--------|-------------------------|
| CREATE | 创建新的候选模型元素。             |
| REVIEW | 置信度不足，进入人工或审批 Agent 确认。 |

4.5 冲突检测与模型补丁确定性规则检测接口路径/字段类型不一致、同一约束阈值不同、

required/provided 接口不兼

容、禁止循环和 P 层能力不满足；NLI 模型或 LLM 判断职责边界、同步/异步策略、架构假设和细化/替代关系。冲突不自动覆盖，而是生成 ConflictRecord。

语义编译器最终输出模型补丁，而不是完整 SysML 文本。确定性适配器再将补丁转换为 SysML v2 AST 或模型 API 操作，并在 proposal 分支上执行语法、类型、引用和 RFLP 一致性检查。

```
[
  {"op": "create_element", "type": "requirement", "temporary_id": "REQ-CAND-12"},
  {"op": "add_alias", "target": "LCOMP-008", "alias": "AccountService"},
  {"op": "create_relation", "relation": "satisfiedBy",
    "source": "REQ-014", "target": "FUNC-021"}
]
```

5. RFLP 模型自动构建

5.1 R 层：需求、约束与验证意图

R 层包含 Requirement、DerivedRequirement、Constraint、Assumption、AcceptanceCriterion 和 VerificationIntent。每条需求必须具有来源、模态、生命周期状态和验收方式；定量要求必须关联 Constraint；Agent 的设计建议不得自动提升为用户需求。

5.2 F 层：从场景与状态变化推导功能

F 层不能通过将需求名词改成动词得到，而应经过 Operational Scenario、State Transition 和 Input/Output Transformation。以版本恢复为例：

```
前置状态：V1、V2、V3 存在，V3 为 active 输入：restore(V1) 状态变化：active_ref 由 V3 切换到 V1 后置状态：V1 为 active，V2 和 V3 仍然保留

功能：
F1 查询版本
F2 校验版本所属关系
F3 修改活动版本引用
F4 保留已有版本记录
F5 记录版本恢复事件
```

Function 采用结构化定义，包含输入、输出、前置条件、后置条件和副作用。F 层检查 R-F 覆盖、输入来源、输出消费者、不可达节点、无终止循环以及实现细节泄漏。

### 5.3 L 层：候选逻辑架构生成

L 层负责功能聚合、状态所有权、事务边界和组件交互，不直接选择框架或数据库。首先构建功能耦合图，边权综合数据共享、调用频率、事务一致性、状态共享、安全域一致性和部署隔离要求。

```
Cij = α·DataSharing
      + β·CallFrequency
      + γ·TransactionConsistency
      + δ·StateSharing
      + ε·SecurityDomain
      - ζ·DeploymentIsolation
```

候选生成结合架构模式实例化与受约束图划分。模式库包括模块化单体、分层架构、微服务、事件驱动、管道—过滤器和多 Agent 协同；约束包括必须同组、禁止同组、状态所有权唯一、事务边界、安全边界、最大组件规模和强制部署隔离。最终保留 2 至 5 套可解释候选。

当功能流跨越逻辑组件边界时自动生成 LogicalInterface，并定义方向、数据类型、同步/异步、幂等性、错误语义、一致性和时序要求。

### 5.4 P 层：具体组件与部署配置

P 层将逻辑组件映射到具体软件模块、数据库、模型服务、中间件和部署节点。组件库中的每个候选必须提供机器可读能力契约，而不是仅保存项目名称和仓库地址。

```
{
  "component": "PostgreSQLVersionRepository",
  "realizes": ["VersionPersistence",
    "TransactionalActiveReferenceUpdate"],
  "provides": ["VersionRepositoryInterface"],
  "requires": ["DatabaseConnection"],
  "quality": {"transactional": true,
    "local_deployment": true},
  "resource": {"memory_mb": 512,
    "gpu_required": false},
  "license": "PostgreSQL",
  "maturity": 0.95 }
```

代码文件和 Git 提交作为 ImplementationArtifact 与 P 层组件关联，不直接等同于 P 层组件。

## 6. P 层组件选择与配置求解

组件选择建模为约束满足和多目标优化问题。设  $x[l,p]$  表示逻辑组件  $l$  是否由物理组件  $p$  实现。硬约束由 CP-SAT 或 SMT 求解器处理，LLM 仅用于补充非结构化描述和解释结果。

### 6.1 硬约束

- 每个逻辑组件至少具有一个物理实现，且全部必要能力被覆盖。
- provided interface 与 required interface 在数据类型、协议和质量约束上兼容。
- 资源总量不超过硬件限制，敏感数据不得离开指定部署域。
- 许可证和组件版本兼容，互斥组件不能同时选择，依赖组件必须一并选择。

6.2 优化目标

- 最小化开发工作量、跨进程接口数量、运行/API 成本和技术风险。
- 最大化组件复用、预期性能、本地可控性和维护成熟度。

离散配置和资源限制适合 OR-Tools CP-SAT；复杂逻辑蕴含、版本兼容以及不可满足原因分析可使用 Z3。

7. 方案权衡与架构基线

Trade Study 的对象是完整的 L/P 组合，不是孤立比较某个数据库或框架。系统不宣称获得绝对全局最优，而是在当前需求、约束、评价指标、权重和证据条件下输出推荐基线。

7.1 三阶段权衡流程

- 1.硬约束过滤：淘汰需求无实现路径、接口不兼容、资源超限、许可证冲突或关键性能低于阈值的方案。
- 2.Pareto 候选：根据开发量、成本、性能、复杂度、风险、可维护性和扩展性保留非支配方案。
- 3.决策分析：使用 TOPSIS 或加权效用排序，并执行权重敏感性分析和不确定性分析。

7.2 指标证据等级

| 等级         | 说明                 |
|------------|--------------------|
| Measured   | 通过基准测试或真实运行得到。     |
| Calculated | 从模型结构或确定性公式计算得到。   |
| Estimated  | 由专家、Agent 或经验区间估计。 |
| Unknown    | 当前无法可靠确定，不应伪装为精确值。 |

对于 API 成本、请求量、模型准确率和开发工时等估计项，可通过 Monte Carlo 采样计算各候选排名概率。Decision 对象需记录候选集合、淘汰原因、指标、权重、证据、关键假设、敏感性分析、批准人和模型版本。

8. 模型驱动开发任务生成

8.1 实际实现模型抽取

第一版限定 Python、FastAPI、Pydantic、PostgreSQL、pytest 和 Docker Compose。采用确定性解析器抽取 Python 模块和类、FastAPI 路由、请求/响应 Schema、模块依赖、数据库表与迁移、测试和部署关系。

8.2 模型差异类型

| Delta 类型 | 含义            | 典型任务                   |
|----------|---------------|------------------------|
| MISSING  | 模型要求存在，代码不存在。 | ImplementationTask     |
| EXTRA    | 代码存在，模型未批准。   | ArchitectureReviewTask |

|             |                 |                    |
|-------------|-----------------|--------------------|
| MISMATCH    | 两侧均存在，但接口或属性不同。 | ContractRepairTask |
| UNVERIFIED  | 实现存在，但缺少验证证据。   | VerificationTask   |
| OUTDATED    | 实现对应旧模型版本。      | MigrationTask      |
| UNALLOCATED | 功能尚未分配逻辑/物理组件。  | LogicalDesignTask  |

8.3 TaskContract

Agent 接收的是机器可执行任务契约，而不是模糊任务标题。TaskContract 包含目标模型元素、读取/修改范围、架构不变量、预期模型变化和验收条件。

```
{
  "task_id": "TASK-017",
  "delta": "MISSING_INTERFACE_IMPLEMENTATION",
  "targets": ["REQ-044", "FUNC-021", "LCOMP-008", "IFACE-019"],
  "read_set": ["backend/auth/models.py",
    "backend/auth/service.py"],
  "write_set": ["backend/auth/routes.py",
    "tests/auth/test_login.py"],
  "invariants": ["UserSchema shall not change",
    "Refresh-token interface remains compatible"],
  "expected_model_delta": [
    "PCOMP-032 implementedBy routes.py",
    "REQ-044 verifiedBy TEST-028"
  ],
  "acceptance": ["OpenAPI conforms to IFACE-019",
    "TEST-028 passes"] }
```

其中 expected\_model\_delta 用于判断 Agent 完成代码后，目标模型状态是否真正得到满足。任务 DAG 依据

DataDefinition → InterfaceDefinition → ComponentImplementation → Integration → Contract Test → RequirementVerification 等模型依赖生成，并检查环依赖、同文件并发修改和未批准基线变更。

9. Agent 执行与模型回写

外层工作流由确定性控制器管理，Agent 只负责需要语言理解或代码生成的步骤。

```
TaskContract
→ 创建 Git 分支或 worktree
→ 生成最小模型切片
→ 准备必要代码上下文
→ 编码 Agent 修改
→ 静态检查
→ 单元测试
→ 契约测试
→ 模型—代码一致性检查
→ Evidence 生成
→ 代码和模型提交
```

| LLM/Agent 负责                         | 确定性程序负责                                 |
|--------------------------------------|-----------------------------------------|
| 复杂语义抽取、低置信度实体对齐、功能/架构候选生成、代码实现、失败解释。 | 模型版本、状态机、任务依赖、权限边界、约束求解、接口比较、测试执行和基线提交。 |

执行完成后生成 Git commit、Code diff、OpenAPI diff、测试日志、覆盖率、性能结果和模型一致性结果。只有代码、测试和模型一致性检查均通过，才更新 P 层实现状态和需求验证状态。若需要修改已批准的 R 层需求或 L 层架构，必须创建 ChangeRequest 并重新进入候选生成和 Trade Study。

10. 原型技术栈与专用模型训练路径

10.1 建议技术栈

| 环节    | 建议实现                                         |
|-------|----------------------------------------------|
| 文档解析  | Markdown AST、python-docx、结构化对话 JSON          |
| 语义抽取  | LLM Structured Output + Pydantic JSON Schema |
| 实体对齐  | BM25 + Embedding + Cross-Encoder             |
| 模型后端  | SysML v2 参考实现与模型 API                         |
| 业务数据  | PostgreSQL; pgvector 用于检索                    |
| 影响分析  | PostgreSQL 递归查询; 后期可增加 Neo4j 只读投影            |
| 约束求解  | OR-Tools CP-SAT + Z3                         |
| 任务编排  | 自定义 DAG 或 LangGraph                          |
| 代码分析  | Python AST、OpenAPI Parser、pytest 结果解析        |
| 版本与证据 | Git、模型 Baseline、Evidence 数据库                 |

10.2 专用 Transformer 训练路径

初期不训练端到端“文档到 RFLP”模型，而是拆分为可独立评价的工程片段分类、原子需求边界识别、工程实体识别、关系抽取、实体对齐和冲突分类。

LLM API 生成候选

→ 人工接受、修改或拒绝

→ 保存映射轨迹

→ 形成监督数据集

→ 微调专用分类器/抽取器/重排器 → LLM 只处理低置信度与复杂语义

工程句分类可采用多标签分类器；实体对齐采用双塔召回加 Cross-Encoder 重排；冲突检测可采用 NLI 模型；隐含需求推理和架构解释继续由 LLM 承担。

11. 实验设计与实施阶段

11.1 对照实验

| 组别  | 开发方式                                 |
|-----|--------------------------------------|
| A 组 | Agent 直接读取规划文档开发。                    |
| B 组 | Agent 通过 RAG 检索规划文档开发。               |
| C 组 | Agent 基于 RFLP 目标模型和 TaskContract 开发。 |

通过 3 至 5 轮受控需求、接口和架构变更，比较三种方式的长期一致性。

11.2 评价指标

| 类别       | 指标                                                                   |
|----------|----------------------------------------------------------------------|
| 语义编译     | 需求识别 P/R/F1、原子拆分准确率、实体类型 F1、关系抽取 F1、实体对齐 Accuracy/MRR、冲突检测 F1、人工确认率。 |
| RFLP 模型  | R-F 覆盖率、F-L 分配完整率、L-P 实现覆盖率、接口闭合率、来源追溯覆盖率、一致性检查通过率。                  |
| 方案选择     | 硬约束满足率、可行候选数量、专家与系统选择一致率、预测/实测误差、排序稳定性。                              |
| Agent 开发 | 一次任务完成率、测试通过率、模型—代码一致率、接口漂移率、重复组件数、架构违规、返工和人工干预次数。                   |

11.3 原型实施阶段

- 4.阶段一：Agent 文档→SemanticClaim→原子需求→实体对齐→冲突检测→SysML v2 R 层。
- 5.阶段二：R 层→场景/状态转换→F 层功能图→功能耦合图→2 至 3 套 L 层候选。
- 6.阶段三：建立 20 至 30 个组件契约，完成 L-P 能力匹配、CP-SAT/Z3 求解、Pareto 权衡和架构基线。
- 7.阶段四：抽取实际实现模型，生成 Delta、TaskContract 和任务 DAG，完成 Agent 执行与 Evidence 回写。

12. 研究范围、创新点与总结

12.1 研究范围控制

第一阶段不实现完整图形化 SysML 编辑器、所有编程语言的代码理解、大规模公共组件市场、完全自动化需求确认、无人工批准的架构基线以及企业级 PLM/权限系统。

- 原型边界：
- 一个软件领域
  - 一个编程语言和固定技术栈
  - 10 至 20 条代表性需求

- 20 至 30 个 P 层候选组件
- 2 至 3 套逻辑架构候选 - 3 至 5 轮受控需求变更

## 12.2 预期创新点

- 面向多 Agent 工程文档的来源感知语义编译，以及 Claim 与权威 ModelElement 分离。
- 通过场景、状态转换和功能流实现 R 到 F 的非直接映射。
- 将 L 层建模为受约束功能图划分和架构模式实例化问题。
- 将 P 层组件选择建模为能力契约匹配和约束优化问题。
- 将 Trade Study 和架构基线纳入 Agent 开发过程。
- 通过目标模型和实际实现模型求差生成具有 expected\_model\_delta 的 TaskContract。
- 建立需求、功能、逻辑架构、物理实现、代码和测试的闭环证据链。

## 12.3 总结

本研究的核心不是让 LLM 生成更多规划文档或整份 SysML 代码，而是在 LLM 与软件代码之间增加一个可追溯、可计算、可约束的系统工程控制层。多源 Agent 文档先被编译为带来源、类型、关系和置信度的语义断言，再通过 RFLP 逐层缩小设计空间；L/P 候选经过约束求解和方案权衡形成架构基线；开发阶段则通过目标模型与实际实现模型求差生成 TaskContract，并以代码、测试和性能 Evidence 回写模型。

多源 Agent 文档  
 → 统一工程语义  
 → RFLP 分层综合  
 → 候选架构与组件配置  
 → 约束求解和方案权衡  
 → 架构基线  
 → 模型差异  
 → Agent 任务契约  
 → 代码与验证  
 → 模型更新

该方法使 AI 辅助开发从基于临时文档和对话上下文的概率性过程，转变为基于统一语义模型、架构基线和验证证据的模型驱动工程过程。

## 参考资料（建议后续正式论文中完善）

- [1] Object Management Group. Systems Modeling Language (SysML) v2 Specification.
- [2] Systems Modeling Community. SysML v2 Reference Implementation and API examples.
- [3] INCOSE. Model-Based Systems Engineering and Digital Engineering guidance.
- [4] Google OR-Tools. CP-SAT Solver Documentation.
- [5] Microsoft Research. Z3 Theorem Prover and Optimization Documentation.
- [6] OMG / ISO related materials on requirements, verification, model interchange and architecture description.